

УО «Белорусский государственный университет информатики и
радиоэлектроники»
Кафедра ПОИТ

Отчет по лабораторной работе №4
по предмету
Распределенные вычисления
Вариант 1

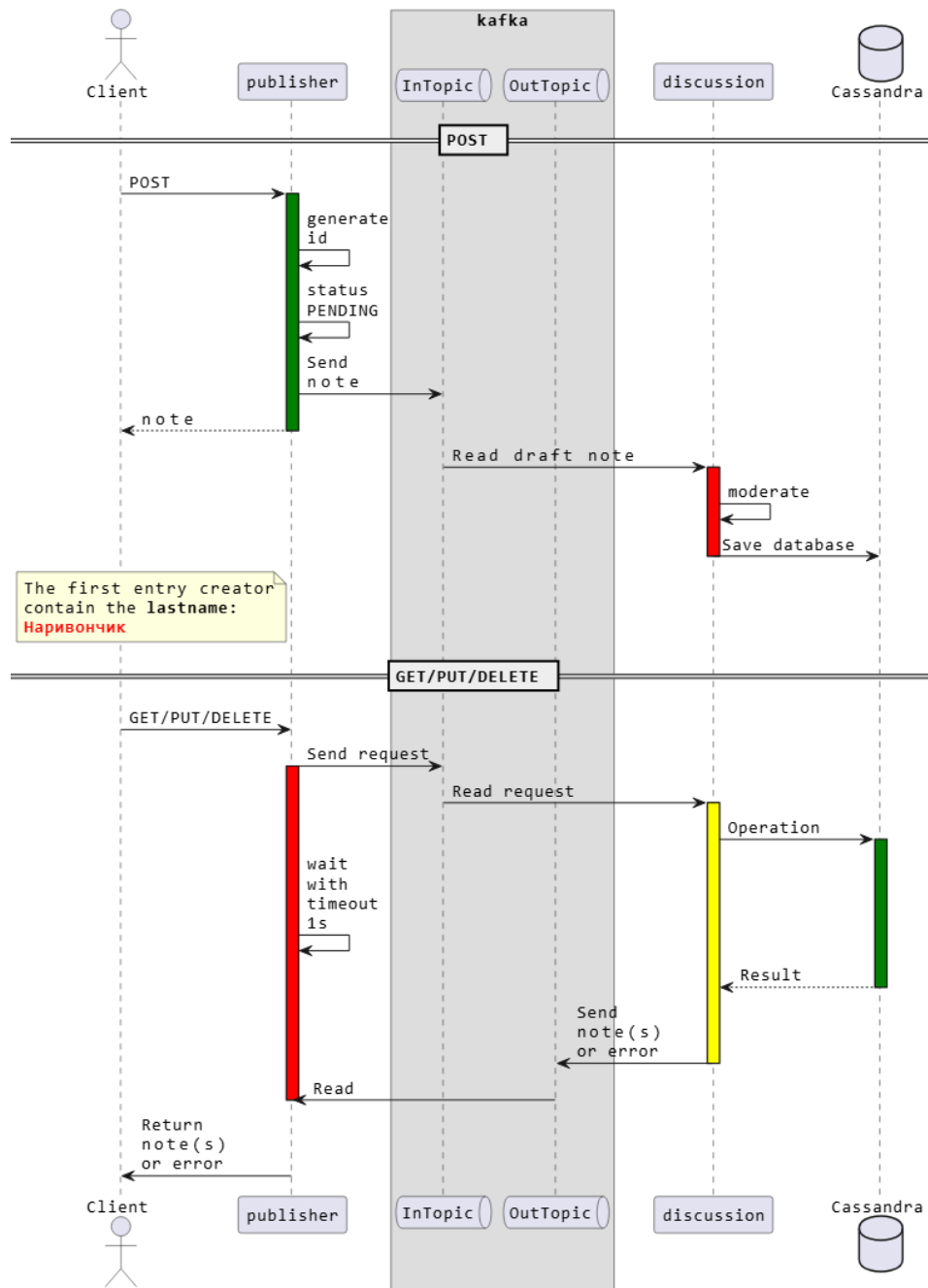
Выполнил:
Наривончик А.М.

Проверил:
Данилова Г.В.

Группа 351004

Минск 2026

Задание



Технические требования

- Используйте префикс **/api/v1.0/** для контроллеров REST и их методов,
- Используйте адрес и порт **localhost:24110** для самого приложения (это модуль **publisher**).
- Не отключайте REST **localhost:24130** для модуля **discussion**.
- Используйте обязательный префикс **tbl_** для таблиц(ы) в базе(ах)

данных

- Используйте подключение по умолчанию к **Kafka** из hub.docker.com
 - Пример запуска:
 - `docker network create kafkanet`
 -
 - `docker run -d --network=kafkanet --name=zookeeper -e \`
 - `ZOOKEEPER_CLIENT_PORT=2181 -e \`
 - `ZOOKEEPER_TICK_TIME=2000 -p 2181:2181 \`
 - `confluentinc/cp-zookeeper`
 -
 - `docker run -d --network=kafkanet --name=kafka -e \`
 - `KAFKA_ZOOKEEPER_CONNECT=zookeeper:2181 -e \`
 - `KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092 -e \`
 - `KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1 -p \`
 - `9092:9092 \`
 - `confluentinc/cp-kafka`

Рекомендации

Подготовительные шаги:

- Изучить документацию по Apache Kafka.
- Установить и настроить Apache Kafka локально, используя docker.
- Создать поле state в сущности **Note** с тремя состояниями PENDING APPROVE DELCINE
- Создать топики **InTopic** и **OutTopic** для обмена сообщениями или автоматизировать создание топиков при их отсутствии (Spring это делает автоматически)

Интеграция Kafka с publisher:

- Создать Kafka Producer в модуле **publisher** для отправки сообщений в топик **InTopic**.
- Настроить отправку **Note** так, чтобы в топике **InTopic** сообщения принадлежащие одной **News** всегда оказывались в одной **partition** (независимо от числа **partition**).
- Создать Kafka Consumer **publisher** для чтения сообщений из **OutTopic**.

Интеграция Kafka с discussion:

- Создать Kafka Consumer в модуле **discussion** для чтения запросов из топика **InTopic**:
 - модуль должен обновлять статус после автоматической модерации на APPROVE или DELCINE
 - алгоритм модерации может быть любым, например на основе стоп-слов

- Создать Kafka Producer в модуле **discussion** для отправки ответов в топик **OutTopic**.

Тестирование и отладка:

- Протестировать передачу сообщений через Kafka.
- Проверить регрессионно работоспособность системы после интеграции Kafka.

Масштабирование и оптимизация:

- Оптимизировать настройки Kafka для повышения производительности.
- Рассмотреть возможность масштабирования Kafka для обеспечения отказоустойчивости.
- Определить минимальное число брокеров, партиций и RF для обеспечения отказоустойчивости кластера Kafka.

Код

```
using Microsoft.AspNetCore.Mvc;
using rest1.application.DTOs.requests;
using rest1.application.DTOs.responses;
using rest1.application.exceptions;
using rest1.application.interfaces.services;

namespace rest1.api.controllers;

[ApiController]
[Route("api/v1.0/[controller]")]
public class CreatorsController : ControllerBase
{
    private readonly ICreatorService _creatorService;
    private readonly ILogger<CreatorsController> _logger;

    public CreatorsController(ICreatorService creatorService,
        ILogger<CreatorsController> logger)
    {
        _creatorService = creatorService;
        _logger = logger;
    }

    [HttpPost]
    [ProducesResponseType(typeof(CreatorResponseTo),
        StatusCodes.Status201Created)]
```

```

[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult<CreatorResponseTo>> CreateCreator(
    [FromBody] CreatorRequestTo createCreatorRequest)
{
    try
    {
        _logger.LogInformation("Creating creator {request}",
createCreatorRequest);

        CreatorResponseTo createdCreator =
            await _creatorService.CreateCreator(createCreatorRequest);

        return CreatedAtAction(
            nameof(CreateCreator),
            new { id = createdCreator.Id },
            createdCreator
        );
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error creating creator");
        return StatusCode(500, "An error occurred while creating the creator");
    }
}

[HttpGet]
[ProducesResponseType(typeof(IEnumerable<CreatorResponseTo>),
StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult<IEnumerable<CreatorResponseTo>>>
GetAllCreators()
{
    try
    {
        _logger.LogInformation("Getting all creators");

        IEnumerable<CreatorResponseTo> creators =
            await _creatorService.GetAllCreators();

```

```

        return Ok(creators);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error getting all creators");
        return StatusCode(500, "An error occurred while retrieving creators");
    }
}

[HttpGet("{id:long}")]
[ProducesResponseType(typeof(CreatorResponseTo),
StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult<CreatorResponseTo>>
GetCreatorById([FromRoute] long id)
{
    try
    {
        _logger.LogInformation("Getting creator by id: {Id}", id);

        var getCreatorRequest = new CreatorRequestTo { Id = id };
        CreatorResponseTo creator =
            await _creatorService.GetCreator(getCreatorRequest);

        return Ok(creator);
    }
    catch (NewNotFoundException ex)
    {
        _logger.LogWarning(ex, "Creator not found with id: {Id}", id);
        return NotFound($"Creator with id {id} not found");
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error getting creator by id: {Id}", id);
        return StatusCode(500, "An error occurred while retrieving the creator");
    }
}

[HttpPut]

```

```

[ProducesResponseType(typeof(CreatorResponseTo),
StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult<CreatorResponseTo>> UpdateCreator(
    [FromBody] CreatorRequestTo updateCreatorRequest)
{
    try
    {
        _logger.LogInformation(
            "Updating creator with id: {Id}",
            updateCreatorRequest.Id);

        var updatedCreator =
            await _creatorService.UpdateCreator(updateCreatorRequest);

        return Ok(updatedCreator);
    }
    catch (NewNotFoundException ex)
    {
        _logger.LogWarning(
            ex,
            "Creator not found for update with id: {Id}",
            updateCreatorRequest.Id);

        return NotFound(
            $"Creator with id {updateCreatorRequest.Id} not found");
    }
    catch (Exception ex)
    {
        _logger.LogError(
            ex,
            "Error updating creator with id: {Id}",
            updateCreatorRequest.Id);

        return StatusCode(
            500,
            "An error occurred while updating the creator");
    }
}

```

```

    }

    [HttpDelete("{id:long}")]
    [ProducesResponseType(StatusCodes.Status204NoContent)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    [ProducesResponseType(StatusCodes.Status500InternalServerError)]
    public async Task<IActionResult> DeleteCreator([FromRoute] long id)
    {
        try
        {
            _logger.LogInformation("Deleting creator with id: {Id}", id);

            var deleteCreatorRequest = new CreatorRequestTo { Id = id };
            await _creatorService.DeleteCreator(deleteCreatorRequest);

            return NoContent();
        }
        catch (CreatorNotFoundException ex)
        {
            _logger.LogWarning(
                ex,
                "Creator not found for deletion with id: {Id}",
                id);

            return NotFound();
        }
        catch (Exception ex)
        {
            _logger.LogError(
                ex,
                "Error deleting creator with id: {Id}",
                id);

            return StatusCode(
                500,
                "An error occurred while deleting the creator");
        }
    }
}

```



```

using Microsoft.AspNetCore.Mvc;
using rest1.application.DTOs.requests;
using rest1.application.DTOs.responses;
using rest1.application.exceptions;
using rest1.application.interfaces.services;

namespace rest1.api.controllers;

[ApiController]
[Route("api/v1.0/[controller]")]
public class MarksController : ControllerBase
{
    private readonly IMarkService _markService;
    private readonly ILogger<MarksController> _logger;

    public MarksController(IMarkService markService, ILogger<MarksController>
logger)
    {
        _markService = markService;
        _logger = logger;
    }

    [HttpPost]
    [ProducesResponseType(typeof(MarkResponseTo),
StatusCodes.Status201Created)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status500InternalServerError)]
    public async Task<ActionResult<MarkResponseTo>> CreateMark(
        [FromBody] MarkRequestTo createMarkRequest)
    {
        try
        {
            _logger.LogInformation("Creating mark {request}", createMarkRequest);

            MarkResponseTo createdMark =
                await _markService.CreateMark(createMarkRequest);

            return CreatedAtAction(
                nameof(CreateMark),
                new { id = createdMark.Id },

```

```

        createdMark
    );
}
catch (Exception ex)
{
    _logger.LogError(ex, "Error creating mark");
    return StatusCode(500, "An error occurred while creating the mark");
}
}

[HttpGet]
[ProducesResponseType(typeof(IEnumerable<MarkResponseTo>),
    StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult<IEnumerable<MarkResponseTo>>>
    GetAllMarks()
{
    try
    {
        _logger.LogInformation("Getting all marks");

        IEnumerable<MarkResponseTo> marks =
            await _markService.GetAllMarks();

        return Ok(marks);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error getting all marks");
        return StatusCode(500, "An error occurred while retrieving marks");
    }
}

[HttpGet("{id:long}")]
[ProducesResponseType(typeof(MarkResponseTo), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult<MarkResponseTo>>
    GetMarkById([FromRoute] long id)
{

```

```

try
{
    _logger.LogInformation("Getting mark by id: {Id}", id);

    var getMarkRequest = new MarkRequestTo { Id = id };
    MarkResponseTo mark =
        await _markService.GetMark(getMarkRequest);

    return Ok(mark);
}
catch (NewNotFoundException ex)
{
    _logger.LogWarning(ex, "Mark not found with id: {Id}", id);
    return NotFound($"Mark with id {id} not found");
}
catch (Exception ex)
{
    _logger.LogError(ex, "Error getting mark by id: {Id}", id);
    return StatusCode(500, "An error occurred while retrieving the mark");
}
}

```

```

[HttpPut]
[ProducesResponseType(typeof(MarkResponseTo), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult<MarkResponseTo>> UpdateMark(
    [FromBody] MarkRequestTo updateMarkRequest)
{
    try
    {
        _logger.LogInformation("Updating mark with id: {Id}",
updateMarkRequest.Id);

        var updatedMark =
            await _markService.UpdateMark(updateMarkRequest);

        return Ok(updatedMark);
    }
}

```

```

catch (NewNotFoundException ex)
{
    _logger.LogWarning(
        ex,
        "Mark not found for update with id: {Id}",
        updateMarkRequest.Id);

    return NotFound(
        $"Mark with id {updateMarkRequest.Id} not found");
}
catch (Exception ex)
{
    _logger.LogError(
        ex,
        "Error updating mark with id: {Id}",
        updateMarkRequest.Id);

    return StatusCode(
        500,
        "An error occurred while updating the mark");
}
}

[HttpDelete("{id:long}")]
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<IActionResult> DeleteMark([FromRoute] long id)
{
    try
    {
        _logger.LogInformation("Deleting mark with id: {Id}", id);

        var deleteMarkRequest = new MarkRequestTo { Id = id };
        await _markService.DeleteMark(deleteMarkRequest);

        return NoContent();
    }
    catch (MarkNotFoundException ex)
    {

```

```

        _logger.LogWarning(ex, "Mark not found for deletion with id: {Id}", id);
        return NotFound();
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error deleting mark with id: {Id}", id);
        return StatusCode(500, "An error occurred while deleting the mark");
    }
}
}

```

```

using Microsoft.AspNetCore.Mvc;
using rest1.application.DTOs.requests;
using rest1.application.DTOs.responses;
using rest1.application.exceptions;
using rest1.application.interfaces.services;

```

```

namespace rest1.api.controllers;

```

```

[ApiController]
[Route("api/v1.0/[controller]")]
public class NewsController : ControllerBase
{
    private readonly INewsService _newsService;
    private readonly ILogger<NewsController> _logger;

    public NewsController(INewsService newsService, ILogger<NewsController>
logger)
    {
        _newsService = newsService;
        _logger = logger;
    }

    [HttpPost]
    [ProducesResponseType(typeof(NewsResponseTo),
StatusCodes.Status201Created)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status500InternalServerError)]
    public async Task<ActionResult<NewsResponseTo>>
CreateNews([FromBody] NewsRequestTo createNewsRequest)

```

```

{
    try
    {
        _logger.LogInformation("Creating news with title: {Title}",
createNewsRequest.Title);

        var createdNews = await _newsService.CreateNews(createNewsRequest);

        return CreatedAtAction(
            nameof(GetNewsById),
            new { id = createdNews.Id },
            createdNews
        );
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error creating news");
        return StatusCode(500, "An error occurred while creating the news");
    }
}

[HttpGet]
[ProducesResponseType(typeof(IEnumerable<NewsResponseTo>),
StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult<IEnumerable<NewsResponseTo>>>
GetAllNews()
{
    try
    {
        _logger.LogInformation("Getting all news");

        var news = await _newsService.GetAllNews();

        return Ok(news);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error getting all news");
        return StatusCode(500, "An error occurred while retrieving news");
    }
}

```

```

    }
}

[HttpGet("{id:long}")]
[ProducesResponseType(typeof(NewsResponseTo), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult<NewsResponseTo>>
GetNewsById([FromRoute] long id)
{
    try
    {
        _logger.LogInformation("Getting news by id: {Id}", id);

        var getNewsRequest = new NewsRequestTo { Id = id };
        var news = await _newsService.GetNews(getNewsRequest);

        return Ok(news);
    }
    catch (NewNotFoundException ex)
    {
        _logger.LogWarning(ex, "News not found with id: {Id}", id);
        return NotFound($"News with id {id} not found");
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error getting news by id: {Id}", id);
        return StatusCode(500, "An error occurred while retrieving the news");
    }
}

[HttpPut]
[ProducesResponseType(typeof(NewsResponseTo), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult<NewsResponseTo>> UpdateNews(
    [FromBody] NewsRequestTo updateNewsRequest)
{
    try

```

```

    {
        _logger.LogInformation("Updating news with id: {Id}",
updateNewsRequest.Id);

        var updatedNews = await
_newsService.UpdateNews(updateNewsRequest);

        return Ok(updatedNews);
    }
    catch (NewNotFoundException ex)
    {
        _logger.LogWarning(ex, "News not found for update with id: {Id}",
updateNewsRequest.Id);
        return NotFound();
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error updating news with id: {Id}",
updateNewsRequest.Id);
        return StatusCode(500, "An error occurred while updating the news");
    }
}

```

```

[HttpDelete("{id:long}")]
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<IActionResult> DeleteNews([FromRoute] long id)
{
    try
    {
        _logger.LogInformation("Deleting news with id: {Id}", id);

        var deleteNewsRequest = new NewsRequestTo { Id = id };
        await _newsService.DeleteNews(deleteNewsRequest);

        return NoContent();
    }
    catch (NewNotFoundException ex)
    {

```



```

        _logger.LogWarning(ex, "News not found for deletion with id: {Id}", id);
        return NotFound();
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error deleting news with id: {Id}", id);
        return StatusCode(500, "An error occurred while deleting the news");
    }
}
}

```

```

using Microsoft.AspNetCore.Mvc;
using rest1.application.DTOs.requests;
using rest1.application.DTOs.responses;
using rest1.application.exceptions;
using rest1.application.interfaces.services;

```

```

namespace rest1.api.controllers;

```

```

[ApiController]
[Route("api/v1.0/[controller]")]
public class NotesController : ControllerBase
{
    private readonly INoteService _noteService;
    private readonly ILogger<NotesController> _logger;

    public NotesController(INoteService noteService, ILogger<NotesController>
logger)
    {
        _noteService = noteService;
        _logger = logger;
    }

    [HttpPost]
    [ProducesResponseType(typeof(NoteResponseTo),
StatusCodes.Status201Created)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status500InternalServerError)]
    public async Task<ActionResult<NoteResponseTo>> CreateNote(
        [FromBody] NoteRequestTo createNoteRequest)
    {

```

```

{
    try
    {
        _logger.LogInformation("Creating note {request}", createNoteRequest);

        NoteResponseTo createdNote =
            await _noteService.CreateNote(createNoteRequest);

        return CreatedAtAction(
            nameof(CreateNote),
            new { id = createdNote.Id },
            createdNote
        );
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error creating note");
        return StatusCode(500, "An error occurred while creating the note");
    }
}

[HttpGet]
[ProducesResponseType(typeof(IEnumerable<NoteResponseTo>),
StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult<IEnumerable<NoteResponseTo>>>
GetAllNotes()
{
    try
    {
        _logger.LogInformation("Getting all notes");

        IEnumerable<NoteResponseTo> notes =
            await _noteService.GetAllNotes();

        return Ok(notes);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error getting all notes");
    }
}

```

```

        return StatusCode(500, "An error occurred while retrieving notes");
    }
}

[HttpGet("{id:long}")]
[ProducesResponseType(typeof(NoteResponseTo),
StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult<NoteResponseTo>>
GetNoteById([FromRoute] long id)
{
    try
    {
        _logger.LogInformation("Getting note by id: {Id}", id);

        var getNoteRequest = new NoteRequestTo { Id = id };
        NoteResponseTo note =
            await _noteService.GetNote(getNoteRequest);

        return Ok(note);
    }
    catch (NewNotFoundException ex)
    {
        _logger.LogWarning(ex, "Note not found with id: {Id}", id);
        return NotFound($"Note with id {id} not found");
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error getting note by id: {Id}", id);
        return StatusCode(500, "An error occurred while retrieving the note");
    }
}

[HttpPut]
[ProducesResponseType(typeof(NoteResponseTo),
StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]

```

```

public async Task<ActionResult<NoteResponseTo>> UpdateNote(
    [FromBody] NoteRequestTo updateNoteRequest)
{
    try
    {
        _logger.LogInformation("Updating note with id: {Id}",
updateNoteRequest.Id);

        var updatedNote =
            await _noteService.UpdateNote(updateNoteRequest);

        return Ok(updatedNote);
    }
    catch (NewNotFoundException ex)
    {
        _logger.LogWarning(
            ex,
            "Note not found for update with id: {Id}",
            updateNoteRequest.Id);

        return NotFound(
            $"Note with id {updateNoteRequest.Id} not found");
    }
    catch (Exception ex)
    {
        _logger.LogError(
            ex,
            "Error updating note with id: {Id}",
            updateNoteRequest.Id);

        return StatusCode(
            500,
            "An error occurred while updating the note");
    }
}

[HttpDelete("{id:long}")]
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]

```

```

public async Task<IActionResult> DeleteNote([FromRoute] long id)
{
    try
    {
        _logger.LogInformation("Deleting note with id: {Id}", id);

        var deleteNoteRequest = new NoteRequestTo { Id = id };
        await _noteService.DeleteNote(deleteNoteRequest);

        return NoContent();
    }
    catch (NoteNotFoundException ex)
    {
        _logger.LogWarning(ex, "Note not found for deletion with id: {Id}", id);
        return NotFound();
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error deleting note with id: {Id}", id);
        return StatusCode(500, "An error occurred while deleting the note");
    }
}

```

```

using System.ComponentModel.DataAnnotations;

```

```

namespace rest1.application.DTOs.requests;

```

```

public class CreatorRequestTo
{
    public long? Id { get; set; }

    [StringLength(64, MinimumLength = 2)]
    public string? Login { get; set; }

    [StringLength(128, MinimumLength = 8)]
    public string? Password { get; set; }

    [StringLength(64, MinimumLength = 2)]
    public string? Firstname { get; set; }
}

```

```

        [StringLength(64, MinimumLength = 2)]
        public string? Lastname { get; set; }
    }

using System.ComponentModel.DataAnnotations;

namespace rest1.application.DTOs.requests;

public class MarkRequestTo
{
    public long? Id { get; set; }

    [StringLength(32, MinimumLength = 2)]
    public string? Name { get; set; }

    public long[] NewsIds { get; set; }
}

using System.ComponentModel.DataAnnotations;

namespace rest1.application.DTOs.requests;

public class NewsRequestTo
{
    public long? Id { get; set; }

    public long CreatorId { get; set; }

    [StringLength(64, MinimumLength = 2)]
    public string Title { get; set; } = string.Empty;

    [StringLength(2048, MinimumLength = 4)]
    public string Content { get; set; } = string.Empty;

    public long[] MarkIds { get; set; }
}

using System.ComponentModel.DataAnnotations;

```

```
namespace rest1.application.DTOs.requests;
```

```
public class NoteRequestTo
{
    public long? Id { get; set; }

    public long NewsId { get; set; }

    [StringLength(2048, MinimumLength = 2)]
    public string Content { get; set; }
}
```

```
namespace rest1.application.DTOs.responses;
```

```
public class CreatorResponseTo (
    string login,
    string password,
    string firstName,
    string lastName)
{
    public long Id { get; set; }

    public string Login { get; set; } = login;

    public string Password { get; set; } = password;

    public string Firstname { get; set; } = firstName;

    public string Lastname { get; set; } = lastName;
}
```

```
namespace rest1.application.DTOs.responses;
```

```
public class MarkResponseTo(
    long id,
    string name)
{
    public long? Id { get; set; } = id;
```

```

    public string? Name { get; set; } = name;
}

namespace rest1.application.DTOs.responses;

public class NewsResponseTo(
    string title,
    string content,
    DateTime createdAt,
    DateTime modified)
{
    public long Id { get; set; }

    public long CreatorId { get; set; }

    public string Title { get; set; } = title;

    public string Content { get; set; } = content;

    public DateTime CreatedAt { get; set; } = createdAt;

    public DateTime Modified { get; set; } = modified;
}

namespace rest1.application.DTOs.responses;

public class NoteResponseTo(
    long id,
    long newsId,
    string content)
{
    public long Id { get; set; } = id;

    public long NewsId { get; set; } = newsId;

    public string Content { get; set; } = content;
}

namespace rest1.application.exceptions;

```



```
public class CreatorNotFoundException : NotFoundException
{
    public CreatorNotFoundException(string message, Exception inner) :
base(message, inner) { }

    public CreatorNotFoundException(string message) : base(message) { }
}
```

```
namespace rest1.application.exceptions;
```

```
public class MarkNotFoundException : NotFoundException
{
    public MarkNotFoundException(string message, Exception inner) :
base(message, inner) { }

    public MarkNotFoundException(string message) : base(message) { }
}
```

```
namespace rest1.application.exceptions;
```

```
public class NewNotFoundException : NotFoundException
{
    public NewNotFoundException(string message, Exception inner) :
base(message, inner) { }

    public NewNotFoundException(string message) : base(message) { }
}
```

```
namespace rest1.application.exceptions;
```

```
public class NotFoundException : Exception
{
    public NotFoundException(string message, Exception inner) : base(message,
inner) { }

    public NotFoundException(string message) : base(message) { }
}
```

```
namespace rest1.application.exceptions;
```

```

public class NoteNotFoundException : NotFoundException
{
    public NoteNotFoundException(string message, Exception inner) :
base(message, inner) { }

    public NoteNotFoundException(string message) : base(message) { }
}

using rest1.core.entities;

namespace rest1.application.interfaces;

public interface ICreatorRepository : IRepository<Creator>
{

}

using rest1.core.entities;

namespace rest1.application.interfaces;

public interface IMarkRepository : IRepository<Mark>
{

}

using rest1.core.entities;

namespace rest1.application.interfaces;

public interface INewsRepository : IRepository<News>
{

}

using rest1.core.entities;

namespace rest1.application.interfaces;

public interface INoteRepository : IRepository<Note>

```

```
{  
  
}
```

```
using rest1.core.entities;
```

```
namespace rest1.application.interfaces;
```

```
public interface IRepository<TEntity> where TEntity : Entity  
{  
    //get records  
    Task<TEntity?> GetByIdAsync(long id, CancellationToken cancellationToken  
= default);  
    Task<IEnumerable<TEntity>> GetAllAsync(CancellationToken ct = default);  
  
    //create record  
    Task<TEntity> AddAsync(TEntity entity, CancellationToken ct = default);  
  
    //update record  
    Task<TEntity?> UpdateAsync(TEntity entity, CancellationToken ct = default);  
  
    //delete record  
    Task<TEntity?> DeleteAsync(TEntity entity, CancellationToken ct = default);  
  
    //find records  
    Task<TEntity?> FindAsync(ISpecification<TEntity> spec, CancellationToken  
ct = default);  
    Task<IEnumerable<TEntity>> FindAllAsync(ISpecification<TEntity> spec,  
CancellationToken ct = default);  
  
    //count records  
    Task<int> CountAsync(ISpecification<TEntity> spec, CancellationToken ct =  
default);  
  
    //is any record  
    Task<bool> AnyAsync(ISpecification<TEntity> spec, CancellationToken ct =  
default);  
}
```

```
using rest1.application.DTOs.requests;
```

```
using rest1.application.DTOs.responses;
```

```
namespace rest1.application.interfaces.services;
```

```
public interface ICreatorService
```

```
{
```

```
    Task<CreatorResponseTo> CreateCreator(CreatorRequestTo  
createCreatorRequestTo);
```

```
    Task<IEnumerable<CreatorResponseTo>> GetAllCreators();
```

```
    Task<CreatorResponseTo> GetCreator(CreatorRequestTo  
getCreatorRequestTo);
```

```
    Task<CreatorResponseTo> UpdateCreator(CreatorRequestTo  
updateCreatorRequestTo);
```

```
    Task DeleteCreator(CreatorRequestTo deleteCreatorRequestTo);  
}
```

```
using rest1.application.DTOs.requests;
```

```
using rest1.application.DTOs.responses;
```

```
namespace rest1.application.interfaces.services;
```

```
public interface IMarkService
```

```
{
```

```
    Task<MarkResponseTo> CreateMark(MarkRequestTo createMarkRequestTo);
```

```
    Task<IEnumerable<MarkResponseTo>> GetAllMarks();
```

```
    Task<MarkResponseTo> GetMark(MarkRequestTo getMarkRequestTo);
```

```
    Task<MarkResponseTo> UpdateMark(MarkRequestTo updateMarkRequestTo);
```

```
    Task DeleteMark(MarkRequestTo deleteMarkRequestTo);  
}
```

```
using rest1.application.DTOs.requests;
```

```
using rest1.application.DTOs.responses;
```

```

namespace rest1.application.interfaces.services;

public interface INewsService
{
    Task<NewsResponseTo> CreateNews(NewsRequestTo createNewsRequestTo);

    Task<IEnumerable<NewsResponseTo>> GetAllNews();

    Task<NewsResponseTo> GetNews(NewsRequestTo getNewsRequestTo);

    Task<NewsResponseTo> UpdateNews(NewsRequestTo
updateNewsRequestTo);

    Task DeleteNews(NewsRequestTo deleteNewsRequestTo);
}

```

```

using rest1.application.DTOs.requests;
using rest1.application.DTOs.responses;

```

```

namespace rest1.application.interfaces.services;

public interface INoteService
{
    Task<NoteResponseTo> CreateNote(NoteRequestTo createNoteRequestTo);

    Task<IEnumerable<NoteResponseTo>> GetAllNotes();

    Task<NoteResponseTo> GetNote(NoteRequestTo getNoteRequestTo);

    Task<NoteResponseTo> UpdateNote(NoteRequestTo updateNoteRequestTo);

    Task DeleteNote(NoteRequestTo deleteNoteRequestTo);
}

```

```

using System.Linq.Expressions;

```

```

namespace rest1.application.interfaces;

```

```

public interface ISpecification<T>

```

```

{
    //criteria expression
    Expression<Func<T, bool>> Criteria { get; }

    //join expressions
    List<Expression<Func<T, object>>> Includes { get; }

    //order by expressions
    Expression<Func<T, object>>? OrderBy { get; }
    Expression<Func<T, object>>? OrderByDescending { get; }

    //taken records count
    int? Take { get; }

    //skipped records count
    int? Skip { get; }

    //pagination flag
    bool IsPagingEnabled { get; }
}

using rest1.application.DTOs.requests;
using rest1.application.DTOs.responses;
using rest1.core.entities;
using AutoMapper;

namespace rest1.application.mappers;

public class CreatorProfile : Profile
{
    public CreatorProfile()
    {
        CreateMap<CreatorRequestTo, Creator>()
            .ForMember(dest => dest.Id, opt => opt.MapFrom(src => src.Id ?? 0))
            .ForMember(dest => dest.Login, opt => opt.MapFrom(src => src.Login))
            .ForMember(dest => dest.Firstname, opt => opt.MapFrom(src =>
src.Firstname))
            .ForMember(dest => dest.Lastname, opt => opt.MapFrom(src =>
src.Lastname))
            .ForMember(dest => dest.Password, opt => opt.MapFrom(src =>

```

```

src.Password));

    CreateMap<Creator, CreatorResponseTo>()
        .ForMember(dest => dest.Id, opt => opt.MapFrom(src => src.Id))
        .ForMember(dest => dest.Login, opt => opt.MapFrom(src => src.Login))
        .ForMember(dest => dest.Firstname, opt => opt.MapFrom(src =>
src.Firstname))
        .ForMember(dest => dest.Lastname, opt => opt.MapFrom(src =>
src.Lastname))
        .ForMember(dest => dest.Password, opt => opt.MapFrom(src =>
src.Password));
    }
}

```

```

using AutoMapper;
using rest1.application.DTOs.requests;
using rest1.application.DTOs.responses;
using rest1.core.entities;

```

```

namespace rest1.application.mappers;

```

```

public class MarkProfile : Profile
{
    public MarkProfile()
    {
        CreateMap<MarkRequestTo, Mark>()
            .ForMember(dest => dest.Id, opt => opt.MapFrom(src => src.Id ?? 0))
            .ForMember(dest => dest.Name, opt => opt.MapFrom(src => src.Name));

        CreateMap<Mark, MarkResponseTo>()
            .ForMember(dest => dest.Id, opt => opt.MapFrom(src => src.Id))
            .ForMember(dest => dest.Name, opt => opt.MapFrom(src => src.Name));
    }
}

```

```

using AutoMapper;
using rest1.application.DTOs.requests;
using rest1.application.DTOs.responses;
using rest1.core.entities;

```

```
namespace rest1.application.mappers;
```

```
public class NewsProfile : Profile
```

```
{  
    public NewsProfile()  
    {  
        CreateMap<NewsRequestTo, News>()  
            .ForMember(dest => dest.Id, opt => opt.MapFrom(src => src.Id ?? 0))  
            .ForMember(dest => dest.CreatedAt, opt => opt.MapFrom(src =>  
DateTime.UtcNow))  
            .ForMember(dest => dest.Modified, opt => opt.MapFrom(src =>  
DateTime.UtcNow))  
            .ForMember(dest => dest.CreatorId, opt => opt.MapFrom(src =>  
src.CreatorId))  
            .ForMember(dest => dest.Content, opt => opt.MapFrom(src =>  
src.Content))  
            .ForMember(dest => dest.Title, opt => opt.MapFrom(src => src.Title));  
  
        CreateMap<News, NewsResponseTo>()  
            .ForMember(dest => dest.Id, opt => opt.MapFrom(src => src.Id))  
            .ForMember(dest => dest.CreatorId, opt => opt.MapFrom(src =>  
src.CreatorId))  
            .ForMember(dest => dest.Title, opt => opt.MapFrom(src => src.Title))  
            .ForMember(dest => dest.Content, opt => opt.MapFrom(src =>  
src.Content))  
            .ForMember(dest => dest.CreatedAt, opt => opt.MapFrom(src =>  
src.CreatedAt))  
            .ForMember(dest => dest.Modified, opt => opt.MapFrom(src =>  
src.Modified));  
    }  
}
```

```
using AutoMapper;  
using rest1.application.DTOs.requests;  
using rest1.application.DTOs.responses;  
using rest1.core.entities;
```

```
namespace rest1.application.mappers;
```

```
public class NoteProfile : Profile
```



```

{
    public NoteProfile()
    {
        CreateMap<NoteRequestTo, Note>()
            .ForMember(dest => dest.Id, opt => opt.MapFrom(src => src.Id ?? 0))
            .ForMember(dest => dest.NewsId, opt => opt.MapFrom(src =>
src.NewsId));

        CreateMap<Note, NoteResponseTo>()
            .ForMember(dest => dest.Id, opt => opt.MapFrom(src => src.Id))
            .ForMember(dest => dest.NewsId, opt => opt.MapFrom(src =>
src.NewsId));
    }
}

```

```

using AutoMapper;
using Microsoft.AspNetCore.Components.Forms;
using rest1.application.DTOs.requests;
using rest1.application.DTOs.responses;
using rest1.application.exceptions;
using rest1.application.interfaces;
using rest1.application.interfaces.services;
using rest1.core.entities;

```

```

namespace rest1.application.services;

```

```

public class CreatorService : ICreatorService
{
    private readonly IMapper _mapper;
    private readonly ICreatorRepository _creatorRepository;

    public CreatorService(IMapper mapper, ICreatorRepository repository)
    {
        _mapper = mapper;
        _creatorRepository = repository;
    }

    public async Task<CreatorResponseTo> CreateCreator(CreatorRequestTo
createCreatorRequestTo)
    {

```

```

        Creator creatorFromDto = _mapper.Map<Creator>(createCreatorRequestTo);

        Creator createdCreator = await
        _creatorRepository.AddAsync(creatorFromDto);

        CreatorResponseTo dtoFromCreatedEditor =
        _mapper.Map<CreatorResponseTo>(createdCreator);

        return dtoFromCreatedEditor;
    }

    public async Task DeleteCreator(CreatorRequestTo deleteCreatorRequestTo)
    {
        Creator creatorFromDto = _mapper.Map<Creator>(deleteCreatorRequestTo);

        _ = await _creatorRepository.DeleteAsync(creatorFromDto)
        ?? throw new CreatorNotFoundException(
            $"Delete creator {creatorFromDto} was not found");
    }

    public async Task<IEnumerable<CreatorResponseTo>> GetAllCreators()
    {
        IEnumerable<Creator> allCreators =
            await _creatorRepository.GetAllAsync();

        var response = new List<CreatorResponseTo>();

        foreach (Creator creator in allCreators)
        {
            response.Add(_mapper.Map<CreatorResponseTo>(creator));
        }

        return response;
    }

    public async Task<CreatorResponseTo> GetCreator(CreatorRequestTo
getCreatorRequestTo)
    {
        Creator creatorFromDto = _mapper.Map<Creator>(getCreatorRequestTo);

```

```

        Creator demandedCreator =
            await _creatorRepository.GetByIdAsync(creatorFromDto.Id)
            ?? throw new CreatorNotFoundException(
                $"Demanded creator {creatorFromDto} was not found");

        return _mapper.Map<CreatorResponseTo>(demandedCreator);
    }

    public async Task<CreatorResponseTo> UpdateCreator(CreatorRequestTo
updateCreatorRequestTo)
    {
        Creator creatorFromDto = _mapper.Map<Creator>(updateCreatorRequestTo);

        Creator updatedCreator =
            await _creatorRepository.UpdateAsync(creatorFromDto)
            ?? throw new CreatorNotFoundException(
                $"Update creator {creatorFromDto} was not found");

        return _mapper.Map<CreatorResponseTo>(updatedCreator);
    }
}

using AutoMapper;
using rest1.application.DTOs.requests;
using rest1.application.DTOs.responses;
using rest1.application.exceptions;
using rest1.application.interfaces;
using rest1.application.interfaces.services;
using rest1.core.entities;

namespace rest1.application.services;

public class MarkService : IMarkService
{
    private readonly IMapper _mapper;
    private readonly IMarkRepository _markRepository;

    public MarkService(IMapper mapper, IMarkRepository repository)
    {
        _mapper = mapper;
    }
}

```

```

    _markRepository = repository;
}

public async Task<MarkResponseTo> CreateMark(MarkRequestTo
createMarkRequestTo)
{
    Mark markFromDto = _mapper.Map<Mark>(createMarkRequestTo);

    Mark createdMark = await _markRepository.AddAsync(markFromDto);

    MarkResponseTo dtoFromCreatedMark =
        _mapper.Map<MarkResponseTo>(createdMark);

    return dtoFromCreatedMark;
}

public async Task DeleteMark(MarkRequestTo deleteMarkRequestTo)
{
    Mark markFromDto = _mapper.Map<Mark>(deleteMarkRequestTo);

    _ = await _markRepository.DeleteAsync(markFromDto)
        ?? throw new MarkNotFoundException(
            $"Delete mark {markFromDto} was not found");
}

public async Task<IEnumerable<MarkResponseTo>> GetAllMarks()
{
    IEnumerable<Mark> allMarks =
        await _markRepository.GetAllAsync();

    var allMarksResponseTos = new List<MarkResponseTo>();

    foreach (Mark mark in allMarks)
    {
        MarkResponseTo markTo =
            _mapper.Map<MarkResponseTo>(mark);

        allMarksResponseTos.Add(markTo);
    }
}

```

```

        return allMarksResponseTos;
    }

    public async Task<MarkResponseTo> GetMark(MarkRequestTo
getMarksRequestTo)
    {
        Mark markFromDto = _mapper.Map<Mark>(getMarksRequestTo);

        Mark demandedMark =
            await _markRepository.GetByIdAsync(markFromDto.Id)
            ?? throw new MarkNotFoundException(
                $"Demanded mark {markFromDto} was not found");

        MarkResponseTo demandedMarkResponseTo =
            _mapper.Map<MarkResponseTo>(demandedMark);

        return demandedMarkResponseTo;
    }

    public async Task<MarkResponseTo> UpdateMark(MarkRequestTo
updateMarkRequestTo)
    {
        Mark markFromDto = _mapper.Map<Mark>(updateMarkRequestTo);

        Mark updateMark =
            await _markRepository.UpdateAsync(markFromDto)
            ?? throw new MarkNotFoundException(
                $"Update mark {markFromDto} was not found");

        MarkResponseTo updateMarkResponseTo =
            _mapper.Map<MarkResponseTo>(updateMark);

        return updateMarkResponseTo;
    }
}

using AutoMapper;
using rest1.application.DTOs.requests;
using rest1.application.DTOs.responses;
using rest1.application.exceptions;

```

```

using rest1.application.interfaces;
using rest1.application.interfaces.services;
using rest1.core.entities;

namespace rest1.application.services;

public class NewsService : INewsService
{
    private readonly IMapper _mapper;

    private readonly INewsRepository _newsRepository;

    public NewsService(IMapper mapper, INewsRepository repository)
    {
        _mapper = mapper;
        _newsRepository = repository;
    }

    public async Task<NewsResponseTo> CreateNews(NewsRequestTo
createNewsRequestTo)
    {
        News newsFromDto = _mapper.Map<News>(createNewsRequestTo);

        News createdNews = await _newsRepository.AddAsync(newsFromDto);

        NewsResponseTo dtoFromCreatedNews =
_mapper.Map<NewsResponseTo>(createdNews);

        return dtoFromCreatedNews;
    }

    public async Task DeleteNews(NewsRequestTo deleteNewsRequestTo)
    {
        News newsFromDto = _mapper.Map<News>(deleteNewsRequestTo);
        _ = await _newsRepository.DeleteAsync(newsFromDto) ?? throw new
NewNotFoundException($"Deleted news {newsFromDto} was not found");
    }

    public async Task<IEnumerable<NewsResponseTo>> GetAllNews()
    {

```

```

IEnumerable<News> allNews = await _newsRepository.GetAllAsync();

var allNewsResponseTos = new List<NewsResponseTo>();
foreach (News news in allNews)
{
    NewsResponseTo newsTo = _mapper.Map<NewsResponseTo>(news);
    allNewsResponseTos.Add(newsTo);
}

return allNewsResponseTos;
}

public async Task<NewsResponseTo> GetNews(NewsRequestTo
getNewsRequestTo)
{
    News newsFromDto = _mapper.Map<News>(getNewsRequestTo);

    News demandedNews = await
_newsRepository.GetByIdAsync(newsFromDto.Id) ?? throw new
NewNotFoundException($"Demanded news {newsFromDto} was not found");

    NewsResponseTo demandedNewsResponseTo =
_mapper.Map<NewsResponseTo>(demandedNews);

    return demandedNewsResponseTo;
}

public async Task<NewsResponseTo> UpdateNews(NewsRequestTo
updateNewsRequestTo)
{
    News newsFromDto = _mapper.Map<News>(updateNewsRequestTo);

    News? updateNews = await _newsRepository.UpdateAsync(newsFromDto)
?? throw new NewNotFoundException($"Update news {newsFromDto} was not
found");

    NewsResponseTo updateNewsResponseTo =
_mapper.Map<NewsResponseTo>(updateNews);

    return updateNewsResponseTo;
}

```

```
}  
}
```

```
using AutoMapper;  
using rest1.application.DTOs.requests;  
using rest1.application.DTOs.responses;  
using rest1.application.exceptions;  
using rest1.application.interfaces;  
using rest1.application.interfaces.services;  
using rest1.core.entities;
```

```
namespace rest1.application.services;
```

```
public class NoteService : INoteService
```

```
{  
    private readonly IMapper _mapper;  
    private readonly INoteRepository _noteRepository;
```

```
    public NoteService(IMapper mapper, INoteRepository repository)  
    {  
        _mapper = mapper;  
        _noteRepository = repository;  
    }
```

```
    public async Task<NoteResponseTo> CreateNote(NoteRequestTo  
createNoteRequestTo)  
    {  
        Note noteFromDto = _mapper.Map<Note>(createNoteRequestTo);  
  
        Note createdNote = await _noteRepository.AddAsync(noteFromDto);  
  
        NoteResponseTo dtoFromCreatedNote =  
            _mapper.Map<NoteResponseTo>(createdNote);  
  
        return dtoFromCreatedNote;  
    }
```

```
    public async Task DeleteNote(NoteRequestTo deleteNoteRequestTo)  
    {  
        Note noteFromDto = _mapper.Map<Note>(deleteNoteRequestTo);
```



```

        _ = await _noteRepository.DeleteAsync(noteFromDto)
        ?? throw new NoteNotFoundException(
            $"Delete note {noteFromDto} was not found");
    }

    public async Task<IEnumerable<NoteResponseTo>> GetAllNotes()
    {
        IEnumerable<Note> allNotes =
            await _noteRepository.GetAllAsync();

        var response = new List<NoteResponseTo>();

        foreach (Note note in allNotes)
        {
            response.Add(_mapper.Map<NoteResponseTo>(note));
        }

        return response;
    }

    public async Task<NoteResponseTo> GetNote(NoteRequestTo
getNoteRequestTo)
    {
        Note noteFromDto = _mapper.Map<Note>(getNoteRequestTo);

        Note demandedNote =
            await _noteRepository.GetByIdAsync(noteFromDto.Id)
            ?? throw new NoteNotFoundException(
                $"Demanded note {noteFromDto} was not found");

        return _mapper.Map<NoteResponseTo>(demandedNote);
    }

    public async Task<NoteResponseTo> UpdateNote(NoteRequestTo
updateNoteRequestTo)
    {
        Note noteFromDto = _mapper.Map<Note>(updateNoteRequestTo);

        Note updatedNote =

```

```

        await _noteRepository.UpdateAsync(noteFromDto)
        ?? throw new NoteNotFoundException(
            $"Update note {noteFromDto} was not found");

        return _mapper.Map<NoteResponseTo>(updatedNote);
    }
}

```

```

using System.Linq.Expressions;
using rest1.application.interfaces;

```

```

namespace rest1.application.specifications;

```

```

public abstract class BaseSpecification<T> : ISpecification<T>
{
    public Expression<Func<T, bool>> Criteria { get; private set; }

    public List<Expression<Func<T, object>>> Includes { get; } = [];

    public Expression<Func<T, object>>? OrderBy { get; private set; }

    public Expression<Func<T, object>>? OrderByDescending { get; private set; }

    public int? Take { get; private set; }

    public int? Skip { get; private set; }

    public bool IsPagingEnabled { get; private set; }

    protected BaseSpecification(Expression<Func<T, bool>> criteria)
    {
        Criteria = criteria;
    }

    protected void AddInclude(Expression<Func<T, object>> includeExpression)
    {
        Includes.Add(includeExpression);
    }

    protected void ApplyPaging(int skip, int take)

```

```

    {
        Skip = skip;
        Take = take;
        IsPagingEnabled = true;
    }

    protected void ApplyOrderBy(Expression<Func<T, object>>
orderByExpression)
    {
        OrderBy = orderByExpression;
    }

    protected void ApplyOrderByDescending(Expression<Func<T, object>>
orderByDescendingExpression)
    {
        OrderByDescending = orderByDescendingExpression;
    }
}

namespace rest1.core.entities;

public class Creator(
    string login,
    string password,
    string firstname,
    string lastname) : Entity
{
    public string Login { get; set; } = login;

    public string Password { get; set; } = password;

    public string Firstname { get; set; } = firstname;

    public string Lastname { get; set; } = lastname;
}

namespace rest1.core.entities;

public abstract class Entity
{

```

```
    public long Id { get; set; }  
}
```

```
namespace rest1.core.entities;
```

```
public class Mark(string name) : Entity  
{  
    public string Name { get; set; } = name;  
    public long[] NewsIds { get; set; } = new long[0];  
}
```

```
namespace rest1.core.entities;
```

```
public class News : Entity  
{  
    public long CreatorId { get; set; }  
  
    public string Title { get; set; } = string.Empty;  
  
    public string Content { get; set; } = string.Empty;  
  
    public DateTime CreatedAt { get; set; }  
  
    public DateTime Modified { get; set; }  
  
    // lookup object  
    public Creator? Creator { get; set; }  
    public long[]? MarkIds { get; set; }  
}
```

```
namespace rest1.core.entities;
```

```
public class Note : Entity  
{  
    public long NewsId { get; set; }  
  
    public string Content { get; set; } = string.Empty;  
  
    // lookup object  
    public News? News { get; set; }  
}
```

```
}
```

```
using rest1.application.interfaces;  
using rest1.core.entities;
```

```
namespace rest1.infrastructure.persistence;
```

```
public class CreatorRepository : Repository<Creator>, ICreatorRepository  
{
```

```
}
```

```
using rest1.application.interfaces;  
using rest1.core.entities;
```

```
namespace rest1.infrastructure.persistence;
```

```
public class MarkRepository: Repository<Mark>, IMarkRepository  
{
```

```
}
```

```
using rest1.application.interfaces;  
using rest1.core.entities;
```

```
namespace rest1.infrastructure.persistence;
```

```
public class NewsRepository: Repository<News>, INewsRepository  
{
```

```
}
```

```
using rest1.application.interfaces;  
using rest1.core.entities;
```

```
namespace rest1.infrastructure.persistence;
```

```
public class NoteRepository: Repository<Note>, INoteRepository
```

```
{  
  
}
```

```
using System.Collections.Concurrent;  
using rest1.application.interfaces;  
using rest1.core.entities;
```

```
namespace rest1.infrastructure.persistence;
```

```
public class Repository<TEntity> : IRepository<TEntity> where TEntity : Entity  
{
```

```
    protected readonly ConcurrentDictionary<long, TEntity> _entities = new();
```

```
    private long _nextId = 1;
```

```
    private readonly Lock _lock = new();
```

```
    public virtual Task<TEntity?> FindAsync(ISpecification<TEntity>  
specification, CancellationToken cancellationToken = default)
```

```
    {  
        cancellationToken.ThrowIfCancellationRequested();
```

```
        lock (_lock)
```

```
        {  
            IQueryable<TEntity> query = ApplySpecification(specification);  
            return Task.FromResult(query.FirstOrDefault());  
        }  
    }
```

```
    public virtual Task<IEnumerable<TEntity>>  
FindAllAsync(ISpecification<TEntity> specification, CancellationToken  
cancellationToken = default)
```

```
    {  
        cancellationToken.ThrowIfCancellationRequested();
```

```
        lock (_lock)
```

```
        {  
            IQueryable<TEntity> query = ApplySpecification(specification);  
            return Task.FromResult(query.ToList().AsEnumerable());  
        }  
    }
```

```
    }  
}
```

```
public virtual Task<int> CountAsync(ISpecification<TEntity> specification,  
CancellationTokens cancellationTokens = default)  
{  
    cancellationTokens.ThrowIfCancellationRequested();  
  
    lock (_lock)  
    {  
        IQueryable<TEntity> query = ApplySpecification(specification);  
        return Task.FromResult(query.Count());  
    }  
}
```

```
public virtual Task<bool> AnyAsync(ISpecification<TEntity> specification,  
CancellationTokens cancellationTokens = default)  
{  
    cancellationTokens.ThrowIfCancellationRequested();  
  
    lock (_lock)  
    {  
        IQueryable<TEntity> query = ApplySpecification(specification);  
        return Task.FromResult(query.Any());  
    }  
}
```

// CRUD

```
public virtual Task<TEntity> AddAsync(TEntity entity, CancellationTokens  
cancellationTokens = default)  
{  
    ArgumentNullException.ThrowIfNull(entity);  
    cancellationTokens.ThrowIfCancellationRequested();  
  
    lock (_lock)  
    {  
        var id = entity.Id;  
        if (id == 0)  
        {
```

```

        id = _nextId;
    }

    if (_entities.TryAdd(_nextId, entity))
    {
        entity.Id = id;
        _nextId++;
        return Task.FromResult(entity);
    }

    throw new InvalidOperationException($"Entity with id {id} already
exists");
}

}

public virtual Task<TEntity?> UpdateAsync(TEntity entity, CancellationToken
cancellationToken = default)
{
    ArgumentNullException.ThrowIfNull(entity);
    cancellationToken.ThrowIfCancellationRequested();

    lock (_lock)
    {
        var id = entity.Id;

        if (_entities.TryGetValue(id, out TEntity? updateEntity))
        {
            _entities[id] = entity;
            updateEntity = _entities[id];
        }

        return Task.FromResult(updateEntity);
    }
}

public virtual Task<TEntity?> DeleteAsync(TEntity entity, CancellationToken
cancellationToken = default)
{
    ArgumentNullException.ThrowIfNull(entity);
    cancellationToken.ThrowIfCancellationRequested();
}

```



```

    lock (_lock)
    {
        var id = entity.Id;
        _entities.TryRemove(id, out TEntity? deletedEntity);

        return Task.FromResult(deletedEntity);
    }
}

public virtual Task<bool> DeleteByIdAsync(long id, CancellationToken
cancellationToken = default)
{
    cancellationToken.ThrowIfCancellationRequested();

    lock (_lock)
    {
        return Task.FromResult(_entities.TryRemove(id, out _));
    }
}

public virtual Task<TEntity?> GetByIdAsync(long id, CancellationToken
cancellationToken = default)
{
    cancellationToken.ThrowIfCancellationRequested();

    lock (_lock)
    {
        return Task.FromResult(_entities.GetValueOrDefault(id));
    }
}

public virtual Task<IEnumerable<TEntity>> GetAllAsync(CancellationToken
cancellationToken = default)
{
    cancellationToken.ThrowIfCancellationRequested();

    lock (_lock)
    {
        return Task.FromResult(_entities.Values.AsEnumerable());
    }
}

```

```

        protected virtual IQueryable<TEntity>
        ApplySpecification(ISpecification<TEntity> spec)
        {
            IQueryable<TEntity> query = _entities.Values.AsQueryable();

            if (spec.Criteria != null)
            {
                query = query.Where(spec.Criteria);
            }
            if (spec.OrderBy != null)
            {
                query = query.OrderBy(spec.OrderBy);
            }
            else if (spec.OrderByDescending != null)
            {
                query = query.OrderByDescending(spec.OrderByDescending);
            }

            if (spec.IsPagingEnabled)
            {
                query = query.Skip(spec.Skip ?? 0)
                    .Take(spec.Take ?? 10);
            }
            return query;
        }
        public void Clear()
        {
            lock (_lock)
            {
                _entities.Clear();
            }
        }
    }
}

```

```

using Microsoft.EntityFrameworkCore;
using rest1.core.entities;
using rest1.persistence.db.configurations;

```

```

namespace rest1.persistence.db;

```

```

public class RestServiceDbContext : DbContext
{
    public RestServiceDbContext(DbContextOptions<RestServiceDbContext>
options)
        : base(options) { }

    public DbSet<Creator> Creators { get; set; }

    public DbSet<Note> Notes { get; set; }

    public DbSet<News> News { get; set; }

    public DbSet<Mark> Marks { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfiguration(new CreatorConfiguration());
        modelBuilder.ApplyConfiguration(new MarkConfiguration());
        modelBuilder.ApplyConfiguration(new NoteConfiguration());
        modelBuilder.ApplyConfiguration(new NewsConfiguration());
        base.OnModelCreating(modelBuilder);
    }
}

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using rest1.core.entities;

namespace rest1.persistence.db.configurations;

public class CreatorConfiguration : IEntityTypeConfiguration<Creator>
{
    public void Configure(EntityTypeBuilder<Creator> builder)
    {
        builder.ToTable("tbl_creator", schema: "public");
        builder.Property(e => e.Id).HasColumnName("id");
        builder.Property(e => e.Login).HasColumnName("login");
        builder.Property(e => e.Password).HasColumnName("password");
        builder.Property(e => e.Firstname).HasColumnName("firstname");
    }
}

```

```

        builder.Property(e => e.Lastname).HasColumnName("lastname");
    }
}

```

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using rest1.core.entities;

```

```

namespace rest1.persistence.db.configurations;

```

```

public class MarkConfiguration : IEntityTypeConfiguration<Mark>
{
    public void Configure(EntityTypeBuilder<Mark> builder)
    {
        builder.ToTable("tbl_mark", schema: "public");
        builder.Property(m => m.Id).HasColumnName("id");
        builder.Property(m => m.Name).HasColumnName("name");
    }
}

```

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using rest1.core.entities;

```

```

namespace rest1.persistence.db.configurations;

```

```

public class NewsConfiguration : IEntityTypeConfiguration<News>
{
    public void Configure(EntityTypeBuilder<News> builder)
    {
        builder.ToTable("tbl_news", schema: "public");
        builder.Property(n => n.Id).HasColumnName("id");
        builder.Property(n => n.CreatorId).HasColumnName("creator_id");
        builder.Property(n => n.Title).HasColumnName("title");
        builder.Property(n => n.Content).HasColumnName("content");
        builder.Property(n => n.CreatedAt).HasColumnName("created");
        builder.Property(n => n.Modified).HasColumnName("modified");
        builder.HasMany(n => n.Marks)
            .WithMany(m => m.News)
            .UsingEntity<Dictionary<string, object>>("tbl_newsMark",

```

```

        j => j.HasOne<Mark>()
            .WithMany()
            .HasForeignKey("MarkId")
            .HasConstraintName("FK_MarkNews_Mark"),
        j => j.HasOne<News>()
            .WithMany()
            .HasForeignKey("NewsId")
            .HasConstraintName("FK_MarkNews_News")
            .onDelete(DeleteBehavior.Cascade),
        j => j.HasKey("MarkId", "NewsId")
    );
}
}

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using rest1.core.entities;

namespace rest1.persistence.db.configurations;

public class NoteConfiguration : IEntityTypeConfiguration<Note>
{
    public void Configure(EntityTypeBuilder<Note> builder)
    {
        builder.ToTable("tbl_note", schema: "public");
        builder.Property(p => p.Id).HasColumnName("id");
        builder.Property(p => p.NewsId).HasColumnName("news_id");
        builder.Property(p => p.Content).HasColumnName("content");
    }
}

using Microsoft.EntityFrameworkCore;
using rest1.application.exceptions.db;
using rest1.application.interfaces;
using rest1.core.entities;

namespace rest1.persistence.db.repositories;

public class DbCreatorRepository : DbRepository<Creator>, ICreatorRepository
{

```

```

    public DbCreatorRepository(RestServiceDbContext dbContext) :
base(dbContext)
    {
    }

    public override async Task<Creator> AddAsync(Creator entity,
CancellationTokentoken = default)
    {
        ArgumentNullException.ThrowIfNull(entity);
        cancellationToken.ThrowIfCancellationRequested();

        var existingEntity = await _dbContext.Where(e => e.Id == entity.Id || e.Login ==
entity.Login).FirstOrDefaultAsync(cancellationToken);

        if (existingEntity != null)
        {
            throw new InvalidOperationException($"Entity with id {entity.Id} already
exists");
        }

        var entry = await _dbContext.AddAsync(entity, cancellationToken);
        try
        {
            await _dbContext.SaveChangesAsync(cancellationToken);
        }
        catch (DbUpdateException ex) when (IsForeignKeyViolation(ex))
        {
            throw new ForeignKeyViolationException("Foreign key doesn't exist");
        }

        return entry.Entity;
    }
}

```

```

using Microsoft.EntityFrameworkCore;
using rest1.application.interfaces;
using rest1.core.entities;

```

```

namespace rest1.persistence.db.repositories;

```

```

public class DbMarkRepository : DbRepository<Mark>, IMarkRepository
{
    public DbMarkRepository(RestServiceDbContext dbContext) : base(dbContext)
    {
    }

    public async Task DeleteMarksWithoutNews()
    {
        var emptyMarkers = await _dbSet.Where(m => m.News.Count ==
0).ToListAsync();
        _dbSet.RemoveRange(emptyMarkers);
        await _dbContext.SaveChangesAsync();
    }
}

```

```

using Microsoft.EntityFrameworkCore;
using rest1.application.exceptions.db;
using rest1.application.interfaces;
using rest1.core.entities;

```

```

namespace rest1.persistence.db.repositories;

```

```

public class DbNewsRepository : DbRepository<News>, INewsRepository
{
    public DbNewsRepository(RestServiceDbContext dbContext) : base(dbContext)
    {
    }

}

```

```

    public override async Task<News> AddAsync(News entity, CancellationToken
cancellation_token = default)
    {

```

```

        ArgumentNullException.ThrowIfNull(entity);
        cancellation_token.ThrowIfCancellationRequested();

```

```

        var existingEntity = await _dbSet.Where(n => n.Id == entity.Id || n.Title ==
entity.Title).FirstOrDefaultAsync(cancellation_token);

```

```

        if (existingEntity != null)

```

```
    {  
        throw new InvalidOperationException($"Entity with id {entity.Id} already  
exists");  
    }
```

```
    var entry = await _dbSet.AddAsync(entity, cancellationToken);  
    try  
    {  
        await _dbContext.SaveChangesAsync(cancellationToken);  
    }  
    catch (DbUpdateException ex) when (IsForeignKeyViolation(ex))  
    {  
        throw new ForeignKeyViolationException("Foreign key doesn't exist");  
    }
```

```
    return entry.Entity;  
}
```

```
public async override Task<News?> DeleteAsync(News entity,  
Cancellation token cancellationToken = default)  
{  
    ArgumentNullException.ThrowIfNull(entity);  
    cancellationToken.ThrowIfCancellationRequested();  
  
    var existingEntity = await _dbSet.Where(n => n.Id == entity.Id)  
        .Include(n => n.Marks)  
        .FirstOrDefaultAsync();  
  
    if (existingEntity == null)  
        return null;  
  
    _dbSet.Remove(existingEntity);  
  
    await _dbContext.SaveChangesAsync(cancellationToken);  
  
    return existingEntity;  
}
```

```
public async Task<News> GetByIdWithMarksAsync(int id)  
{
```



```

        return await _dbSet
            .Include(n => n.Marks)
            .FirstOrDefaultAsync(n => n.Id == id);
    }

    public async Task RemoveAsync(News news)
    {
        _dbSet.Remove(news);
        await _dbContext.SaveChangesAsync();
    }
}

using rest1.application.interfaces;
using rest1.core.entities;

namespace rest1.persistence.db.repositories;

public class DbNoteRepository : DbRepository<Note>, INoteRepository
{
    public DbNoteRepository(RestServiceDbContext dbContext) : base(dbContext)
    {
    }
}

using Microsoft.EntityFrameworkCore;
using Npgsql;
using rest1.application.exceptions.db;
using rest1.application.interfaces;
using rest1.core.entities;

namespace rest1.persistence.db.repositories;

public class DbRepository<TEntity> : IRepository<TEntity>
    where TEntity : Entity
{
    protected readonly DbSet<TEntity> _dbSet;
    protected readonly DbContext _dbContext;

    public DbRepository(RestServiceDbContext dbContext)

```

```

    {
        _dbContext = dbContext ?? throw new
ArgumentNullException(nameof(dbContext));
        _dbSet = dbContext.Set<TEntity>();
    }

    public virtual async Task<TEntity?> FindAsync(ISpecification<TEntity>
specification, CancellationToken cancellationToken = default)
    {
        cancellationToken.ThrowIfCancellationRequested();

        IQueryable<TEntity> query = ApplySpecification(specification);
        return await query.FirstOrDefaultAsync(cancellationToken);
    }

    public virtual async Task<IEnumerable<TEntity>>
FindAllAsync(ISpecification<TEntity> specification, CancellationToken
cancellationToken = default)
    {
        cancellationToken.ThrowIfCancellationRequested();

        IQueryable<TEntity> query = ApplySpecification(specification);
        return await query.ToListAsync(cancellationToken);
    }

    public virtual async Task<int> CountAsync(ISpecification<TEntity>
specification, CancellationToken cancellationToken = default)
    {
        cancellationToken.ThrowIfCancellationRequested();

        IQueryable<TEntity> query = ApplySpecification(specification);
        return await query.CountAsync(cancellationToken);
    }

    public virtual async Task<bool> AnyAsync(ISpecification<TEntity>
specification, CancellationToken cancellationToken = default)
    {
        cancellationToken.ThrowIfCancellationRequested();

        IQueryable<TEntity> query = ApplySpecification(specification);

```

```

        return await query.AnyAsync(cancellationToken);
    }

    // CRUD
    public virtual async Task<TEntity> AddAsync(TEntity entity,
CancellationTokn cancellationTokn = default)
    {
        ArgumentNullException.ThrowIfNull(entity);
        cancellationTokn.ThrowIfCancellationRequested();

        var existingEntity = await _dbSet.FindAsync([entity.Id],
cancellationTokn: cancellationTokn);

        if (existingEntity != null)
        {
            throw new InvalidOperationException($"Entity with id {entity.Id}
already exists");
        }

        var entry = await _dbSet.AddAsync(entity, cancellationTokn);
        try
        {
            await _dbContext.SaveChangesAsync(cancellationTokn);
        }
        catch (DbUpdateException ex) when (IsForeignKeyViolation(ex))
        {
            throw new ForeignKeyViolationException("Foreign key doesn't exist");
        }

        return entry.Entity;
    }

    protected static bool IsForeignKeyViolation(DbUpdateException ex)
    {
        return ex.InnerException switch
        {
            {
                PostgresException pgEx => pgEx.SqlState == "23503",
                _ => false
            };
        }
    }

```

```

    public virtual async Task<TEntity?> UpdateAsync(TEntity entity,
CancellationTokentoken = default)
    {
        ArgumentNullException.ThrowIfNull(entity);
        cancellationToken.ThrowIfCancellationRequested();

        var existingEntity = await _dbSet.FindAsync([entity.Id,
cancellationToken], cancellationToken: cancellationToken);

        if (existingEntity == null)
            return null;

        _dbContext.Entry(existingEntity).CurrentValues.SetValues(entity);
        await _dbContext.SaveChangesAsync(cancellationToken);

        return existingEntity;
    }

    public virtual async Task<TEntity?> DeleteAsync(TEntity entity,
CancellationTokentoken = default)
    {
        ArgumentNullException.ThrowIfNull(entity);
        cancellationToken.ThrowIfCancellationRequested();

        var existingEntity = await _dbSet.FindAsync(entity.Id, cancellationToken);

        if (existingEntity == null)
            return null;

        _dbSet.Remove(existingEntity);
        await _dbContext.SaveChangesAsync(cancellationToken);

        return existingEntity;
    }

    public virtual async Task<bool> DeleteByIdAsync(long id,
CancellationTokentoken = default)
    {
        cancellationToken.ThrowIfCancellationRequested();

```

```

        var entity = await _dbSet.FindAsync(id , cancellationToken);

        if (entity == null)
            return false;

        _dbSet.Remove(entity);
        await _dbContext.SaveChangesAsync(cancellationToken);

        return true;
    }

    public virtual async Task<TEntity?> GetByIdAsync(long id,
CancellationTokentoken cancellationToken = default)
    {
        cancellationToken.ThrowIfCancellationRequested();

        return await _dbSet.FindAsync(id, cancellationToken);
    }

    public virtual async Task<IEnumerable<TEntity>>
GetAllAsync(CancellationTokentoken cancellationToken = default)
    {
        cancellationToken.ThrowIfCancellationRequested();

        return await _dbSet.ToListAsync(cancellationToken);
    }

    protected virtual IQueryable<TEntity>
ApplySpecification(ISpecification<TEntity> spec)
    {
        IQueryable<TEntity> query = _dbSet.AsQueryable();

        if (spec.Criteria != null)
        {
            query = query.Where(spec.Criteria);
        }

        // Применяем инклюды (включаемые навигационные свойства)
        query = spec.Includes

```

```

        .Aggregate(query, (current, include) => current.Include(include));

    if (spec.OrderBy != null)
    {
        query = query.OrderBy(spec.OrderBy);
    }
    else if (spec.OrderByDescending != null)
    {
        query = query.OrderByDescending(spec.OrderByDescending);
    }

    if (spec.IsPagingEnabled)
    {
        query = query.Skip(spec.Skip ?? 0)
            .Take(spec.Take ?? 10);
    }

    return query;
}
}

```

```

using Cassandra;
using Cassandra.Mapping;

```

```

namespace DiscussionModule.persistence;

```

```

public class CassandraContext
{
    private readonly ICluster _cluster;
    private Cassandra.ISession _session;
    private IMapper _mapper;

    public CassandraContext(IConfiguration configuration)
    {
        var connectionString =
configuration.GetConnectionString("CassandraConnection");
        var options = new CassandraOptions(connectionString);

        _cluster = Cluster.Builder()
            .AddContactPoint(options.Host)

```

```

        .WithPort(options.Port)
        .Build();

// Сначала подключаемся без указания keyspace
_session = _cluster.Connect();
}

public IMapper Mapper => _mapper;
public Cassandra.ISession Session => _session;

public void CreateKeyspaceIfNotExists(string keyspaceName)
{
    var createKeyspaceQuery = $"@\"
        CREATE KEYSPACE IF NOT EXISTS {keyspaceName}
        WITH replication = {{
            'class': 'SimpleStrategy',
            'replication_factor': 1
        }}\";

    _session.Execute(createKeyspaceQuery);

    // Переключаемся на созданный keyspace
    _session.ChangeKeyspace(keyspaceName);

    // Создаем mapper после переключения keyspace
    _mapper = new Mapper(_session);
}

public void CreateTableIfNotExists()
{
    var createTableQuery = @"
        CREATE TABLE IF NOT EXISTS tbl_notes (
            id bigint PRIMARY KEY,
            news_id bigint,
            content text,
            country text
        )\";

    _session.Execute(createTableQuery);
}

```

```

public void CreateCounterTableIfNotExists()
{
    _session.Execute("DROP TABLE IF EXISTS tbl_counters");
    var createCounterTableQuery = @"
CREATE TABLE IF NOT EXISTS tbl_counters (
    counter_name text PRIMARY KEY,
    counter_value counter
)";

    _session.Execute(createCounterTableQuery);

    var initCounterQuery = @"
UPDATE tbl_counters
SET counter_value = counter_value + 0
WHERE counter_name = 'note_id'";

    _session.Execute(initCounterQuery);
}
}

public class CassandraOptions
{
    public string Host { get; }
    public int Port { get; }
    public string Keyspace { get; }

    public CassandraOptions(string connectionString)
    {
        // Парсинг connection string формата:
        jdbc:cassandra://localhost:9042/distcompcassandra
        var uri = new Uri(connectionString.Replace("jdbc:", ""));
        Host = uri.Host;
        Port = uri.Port;
        Keyspace = uri.AbsolutePath.TrimStart('/');
    }
}

using Cassandra;
using Cassandra.Mapping;

```



```

using DiscussionModule.interfaces;
using DiscussionModule.models;

namespace DiscussionModule.persistence.repositories;

public class NoteRepository : INoteRepository
{
    private readonly IMapper _mapper;
    private readonly Cassandra.ISession _session;

    public NoteRepository(CassandraContext context)
    {
        _mapper = context.Mapper;
        _session = context.Session;
    }

    public async Task<Note> AddAsync(Note note)
    {
        note.Id = await GetNextId();

        await _mapper.InsertAsync(note);
        return note;
    }

    public async Task<Note?> GetByIdAsync(long id)
    {
        var note = await _mapper.SingleOrDefaultAsync<Note>(
            "WHERE id = ?", id);
        return note;
    }

    public async Task<List<Note>> GetAllAsync()
    {
        var notes = await _mapper.FetchAsync<Note>();
        return notes.ToList();
    }

    public async Task<Note?> UpdateAsync(Note note)
    {
        var existing = await GetByIdAsync(note.Id);

```

```

        if (existing == null)
            return null;

        await _mapper.UpdateAsync(note);
        return note;
    }

    public async Task DeleteAsync(Note note)
    {
        await _mapper.DeleteAsync(note);
    }

    private async Task<long> GetNextId()
    {
        try
        {
            var updateQuery = @"
                UPDATE tbl_counters
                SET counter_value = counter_value + 1
                WHERE counter_name = 'note_id'";

            await _session.ExecuteAsync(new SimpleStatement(updateQuery));

            var selectQuery = @"
                SELECT counter_value FROM tbl_counters
                WHERE counter_name = 'note_id'";

            var row = (await _session.ExecuteAsync(
                new SimpleStatement(selectQuery))).FirstOrDefault();

            if (row != null)
            {
                return row.GetValue<long>("counter_value");
            }

            var initQuery = @"
                UPDATE tbl_counters
                SET counter_value = counter_value + 0
                WHERE counter_name = 'note_id'";

```

```

        await _session.ExecuteAsync(new SimpleStatement(initQuery));

        var initRow = (await _session.ExecuteAsync(
            new SimpleStatement(selectQuery))).FirstOrDefault();

        return initRow?.GetValue<long>("counter_value") ?? 0;
    }
    catch (Exception ex)
    {
        throw new Exception("Ошибка при генерации ID для заметки", ex);
    }
}

```

```

using DiscussionModule.DTOs.requests;
using DiscussionModule.DTOs.responses;
using DiscussionModule.interfaces;
using Microsoft.AspNetCore.Mvc;

```

```

namespace DiscussionModule.controllers;

```

```

[ApiController]
[Route("api/v1.0/[controller]")]
public class NotesController : ControllerBase
{
    private readonly INoteService _service;

    public NotesController(INoteService service)
    {
        _service = service;
    }

    [HttpPost]
    public async Task<ActionResult<NoteResponseTo>> CreateNote([FromBody]
        NoteRequestTo note)
    {
        var created = await _service.CreateNote(note);
        return CreatedAtAction(nameof(GetNoteById), new { id = created.Id },
            created);
    }
}

```

```

[HttpGet]
public async Task<ActionResult<IEnumerable<NoteResponseTo>>>
GetAllNotes()
{
    try
    {
        var notes = await _service.GetAllNotes();
        return Ok(notes);
    }
    catch(Exception)
    {
        return Ok();
    }
}

[HttpGet("{id:long}")]
public async Task<ActionResult<NoteResponseTo>> GetNoteById(long id)
{
    var dto = new NoteRequestTo() { Id = id };
    var post = await _service.GetNote(dto);
    if (post == null)
        return NotFound();

    return Ok(post);
}

[HttpPut("{id:long}")]
public async Task<ActionResult<NoteResponseTo>> UpdateNote(long id,
[FromBody] NoteRequestTo note)
{
    note.Id = id;
    var updated = await _service.UpdateNote(note);
    if (updated == null)
        return NotFound();

    return Ok(updated);
}

[HttpDelete("{id:long}")]

```

```

public async Task<IActionResult> DeleteNote(long id)
{
    var dto = new NoteRequestTo() { Id = id };
    await _service.DeleteNote(dto);

    return NoContent();
}
}

```

```

using Cassandra.Mapping.Attributes;

```

```

namespace DiscussionModule.models;

```

```

[Table("tbl_notes")]
public class Note
{
    [PartitionKey]
    [Column("id")]
    public long Id { get; set; }

    [Column("news_id")]
    public long NewsId { get; set; }

    [Column("content")]
    public string Content { get; set; } = string.Empty;

    [Column("country")]
    public string Country { get; set; } = string.Empty;
}

```

```

using Confluent.Kafka;
using System.Text.Json;
using DiscussionModule.interfaces;
using DiscussionModule.DTOs.requests;
using AutoMapper;

```

```

namespace DiscussionModule.kafka;

```

```

public class KafkaConsumer : BackgroundService
{

```

```
private readonly IConsumer<string, string> _consumer;
private readonly IServiceProvider _serviceProvider;
private readonly string _inTopic;
private readonly KafkaProducer _producer;
private readonly List<string> _stopWords = new() { "spam", "bad", "offensive",
"xxx" };
```

```
public KafkaConsumer(IServiceProvider serviceProvider, string
bootstrapServers, string inTopic, string outTopic)
{
    var config = new ConsumerConfig
    {
        BootstrapServers = bootstrapServers,
        GroupId = "discussion-consumer-group",
        AutoOffsetReset = AutoOffsetReset.Earliest,
        EnableAutoCommit = true
    };

    _consumer = new ConsumerBuilder<string, string>(config).Build();
    _serviceProvider = serviceProvider;
    _inTopic = inTopic;
    _producer = new KafkaProducer(bootstrapServers, outTopic);
}
```

```
protected override async Task ExecuteAsync(CancellationToken
stoppingToken)
{
    _consumer.Subscribe(_inTopic);
    Console.WriteLine($"[Kafka] Consumer started, listening to topic:
{_inTopic}");
```

```
while (!stoppingToken.IsCancellationRequested)
{
    try
    {
        var consumeResult = _consumer.Consume(stoppingToken);

        if (consumeResult != null)
        {
            await ProcessMessageAsync(consumeResult.Message.Key,
```

```

consumeResult.Message.Value);
    }
}
catch (OperationCanceledException)
{
    break;
}
catch (Exception ex)
{
    Console.WriteLine($"[Kafka] Error: {ex.Message}");
}
}

_consumer.Close();
}

private async Task ProcessMessageAsync(string key, string jsonMessage)
{
    using var scope = _serviceProvider.CreateScope();
    var noteService =
scope.ServiceProvider.GetRequiredService<INoteService>();
    var mapper = scope.ServiceProvider.GetRequiredService<IMapper>();
    var operation = key;
    var noteRequest = JsonSerializer.Deserialize<NoteRequestTo>(jsonMessage);

    try
    {
        object response = null;
        string status = "SUCCESS";

        switch (operation.ToUpper())
        {
            case "CREATE":
                if (await ModerateNoteAsync(noteRequest))
                {
                    var createdNote = await noteService.CreateNote(noteRequest);
                    response = createdNote;
                    status = "APPROVED";
                }
                else

```

```

        {
            status = "DECLINED";
            response = new { Message = "Note declined by moderation" };
        }
        break;

    case "GET":
        response = await noteService.GetNote(noteRequest);
        break;

    case "UPDATE":
        response = await noteService.UpdateNote(noteRequest);
        break;

    case "DELETE":
        await noteService.DeleteNote(noteRequest);
        response = new { Message = "Deleted successfully" };
        break;
    }

    var responseMessage = new
    {
        Operation = operation,
        Status = status,
        Data = response,
        RequestId = noteRequest?.Id ?? 0
    };

    await _producer.SendMessageAsync(noteRequest?.Id.ToString() ?? "0",
responseMessage);
    }
    catch (Exception ex)
    {
        var errorResponse = new
        {
            Operation = key,
            Status = "ERROR",
            Error = ex.Message,
            RequestId = noteRequest?.Id ?? 0
        };
    }

```



```

        await _producer.SendMessageAsync(noteRequest?.Id.ToString() ?? "0",
errorResponse);
        Console.WriteLine($"[Kafka] Error processing: {ex.Message}");
    }
}

```

```

private Task<bool> ModerateNoteAsync(NoteRequestTo note)
{

```

```

    // Проверка стоп-слов
    foreach (var stopWord in _stopWords)
    {
        if (note.Content.ToLower().Contains(stopWord))
        {
            return Task.FromResult(false);
        }
    }

```

```

    return Task.FromResult(true);
}

```

```

public override void Dispose()
{
    _consumer?.Dispose();
    _producer?.Dispose();
    base.Dispose();
}
}

```

```

using Confluent.Kafka;
using System.Text.Json;
using Confluent.Kafka.Admin;

```

```

namespace DiscussionModule.kafka;

```

```

public class KafkaProducer : IDisposable
{
    private readonly IProducer<string, string> _producer;
    private readonly string _outTopic;

```

```

private readonly string _bootstrapServers;

public KafkaProducer(string bootstrapServers, string outTopic)
{
    _bootstrapServers = bootstrapServers;
    var config = new ProducerConfig
    {
        BootstrapServers = bootstrapServers,
        ClientId = "discussion-module-producer",
        Acks = Acks.All,
        EnableIdempotence = true
    };

    _producer = new ProducerBuilder<string, string>(config).Build();
    _outTopic = outTopic;
}

public async Task SendMessageAsync(string key, object message)
{
    try
    {
        var jsonMessage = JsonSerializer.Serialize(message);
        var result = await _producer.ProduceAsync(_outTopic, new
Message<string, string>
        {
            Key = key,
            Value = jsonMessage
        });

    }
    catch (ProduceException<string, string> ex)
    {
        Console.WriteLine($"[Kafka] Error: {ex.Error.Reason}");
        throw;
    }
}

public async Task EnsureTopicsExistAsync(params string[] topics)
{
    try

```

```

{
    var config = new AdminClientConfig
    {
        BootstrapServers = _bootstrapServers
    };

    using var adminClient = new AdminClientBuilder(config).Build();

    var topicSpecifications = topics.Select(topic => new TopicSpecification
    {
        Name = topic,
        NumPartitions = 3,
        ReplicationFactor = 1
    }).ToList();

    try
    {
        await adminClient.CreateTopicsAsync(topicSpecifications);
        Console.WriteLine($"[Kafka] Topics created: {string.Join(", ",
topics)}}");
    }
    catch (CreateTopicsException ex)
    {
        if (!ex.Results.Any(r => r.Error.Code !=
ErrorCode.TopicAlreadyExists))
        {
            Console.WriteLine("[Kafka] Topics already exist");
        }
        else
        {
            Console.WriteLine($"[Kafka] Error: {ex.Message}");
        }
    }
}
catch (Exception ex)
{
    Console.WriteLine($"[Kafka] Warning: {ex.Message}");
}
}

```

```

    public void Dispose()
    {
        _producer?.Dispose();
    }
}

using DiscussionModule.interfaces;
using DiscussionModule.kafka;
using DiscussionModule.mappers;
using DiscussionModule.persistence;
using DiscussionModule.persistence.repositories;
using DiscussionModule.services;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container
builder.Services.AddControllers();

// Register CassandraContext as singleton
builder.Services.AddSingleton<CassandraContext>(serviceProvider =>
{
    var configuration = serviceProvider.GetRequiredService<IConfiguration>();
    var context = new CassandraContext(configuration);

    context.CreateKeyspaceIfNotExists("distcompcassandra");
    context.CreateTableIfNotExists();
    context.CreateCounterTableIfNotExists();

    return context;
});

builder.Services.AddScoped<INoteRepository, NoteRepository>();
builder.Services.AddScoped<INoteService, NoteService>();

// AutoMapper
builder.Services.AddAutoMapper(config =>
{
    config.AddProfile<NoteProfile>();
});

```

```

// === KAFKA CONFIGURATION ===
var kafkaEnabled = builder.Configuration.GetValue<bool>("Kafka:Enabled");

if (kafkaEnabled)
{
    var kafkaBootstrapServers =
builder.Configuration.GetValue<string>("Kafka:BootstrapServers") ??
"localhost:9092";
    var kafkaInTopic = builder.Configuration.GetValue<string>("Kafka:InTopic")
?? "InTopic";
    var kafkaOutTopic =
builder.Configuration.GetValue<string>("Kafka:OutTopic") ?? "OutTopic";

    builder.Services.AddSingleton<KafkaProducer>(provider =>
        new KafkaProducer(kafkaBootstrapServers, kafkaOutTopic));

    builder.Services.AddSingleton<KafkaConsumer>(provider =>
        new KafkaConsumer(
            provider,
            kafkaBootstrapServers,
            kafkaInTopic,
            kafkaOutTopic
        ));

    builder.Services.AddHostedService(provider =>
        provider.GetRequiredService<KafkaConsumer>());
}

var app = builder.Build();

if (kafkaEnabled)
{
    using var scope = app.Services.CreateScope();
    var producer = scope.ServiceProvider.GetRequiredService<KafkaProducer>();

    await producer.EnsureTopicsExistAsync(
        builder.Configuration["Kafka:InTopic"],
        builder.Configuration["Kafka:OutTopic"]
    );
}

```

```
app.MapControllers();  
app.Run();
```